Fin-Comply DPDP Engine
Research-Backed Remediation Report

Proposed remediation for Issues E-01 through E-13

Purpose	Document the proposed fixes, implementation direction, dependencies, and verification criteria for the current system issues.
Scope	Data ingestion, synthetic data generation, rule evaluation, risk scoring, ML training/evaluation, API reporting, security, and governance.
Status	Proposed plan — not yet implemented. Research-backed principles are distinguished from project-specific design choices.
Example tenant	QUICKLOAN-001 is used throughout to make the issues concrete.

Precision note: The research supports the engineering principles. It does not automatically validate project-specific score thresholds such as 65/45, and the cited legal sources should not be treated as legal advice. Indian DPDP legal claims must be checked against authoritative legislation/rules.

1. Executive Summary

The system has problems at several connected layers. The safest remediation is therefore not to patch everything at once. First make the database truthful; then make generated test data reproducible; then fix rule evidence and scoring; only after that retrain the ML model; finally complete the API, security, and governance layers.

- Bad input data can produce bad rule findings, bad risk scores, and bad ML training.
- Missing evidence cannot be reliably reconstructed later by the API.
- A missing ML response can make the system compute one answer and display another.
- An unsecured rule-approval endpoint can allow an attacker to change the logic used to judge every tenant.

Strict order

1	Ingestion integrity	Stop invented records and stale tenant state.
2	Generator reconciliation	One deterministic, profile-driven source of test data.
3	Clean regeneration + re-ingestion	Remove contamination from previous runs.
4	Rule/evidence fixes	Make every finding traceable.
5	Risk scoring	Use the rule's true opportunity set.
6	ML retraining	Train/evaluate only on clean, leakage-free data.
7	API/explainability	Expose ML output and real SHAP separately.
8	Security/governance	Protect rule changes and version results.

2. Foundational Principle — Fix Data Before Models

Think of the system as a chain. If the database contains invented or stale facts, downstream calculations can be perfectly coded and still be wrong. Data integrity must therefore come before score recalibration and ML retraining.

> **QuickLoan example:** A transaction refers to consent_hash C123, but C123 is missing. If ingestion creates an expired consent record automatically, the rule engine sees a violation that was never present in the uploaded source. Training on that dataset teaches the ML model the ingestion bug.

The remediation is consequently designed as a dependency chain rather than thirteen unrelated code edits.

3. Detailed Issue-by-Issue Remediation

Issue 1: E-04 — Manufactured consent placeholders and false third-party DPAs

Layer: Data ingestion

Current problem

Missing consent_hash or third_party_hash references are currently handled by manufacturing related records, including an expired consent or a third-party system with dpa_signed=False.

QuickLoan example

> QuickLoan sends consent_hash=C123, but C123 is absent. The system creates C123 as expired, so QuickLoan now appears to have an invalid consent that was never supplied.

Why it matters

Invented facts contaminate findings, scores, training data, and audit evidence.

Remediation being considered

Reject and report the affected row/reference instead of creating a parent record. Validate the incoming dataset before committing it.

Verification

Test missing-reference cases and confirm no phantom records are created.

Research basis [R1] Kleppmann, Designing Data-Intensive Applications; [R2] PostgreSQL INSERT/constraint behavior.

Issue 2: E-10 — Stale data accumulation from ON CONFLICT DO NOTHING

Layer: Ingestion / tenant isolation

Current problem

DO NOTHING prevents duplicate-key errors but can leave old values untouched when a tenant uploads corrected data.

QuickLoan example

> QuickLoan changes purpose from marketing to loan_servicing. If the existing key conflicts and DO NOTHING is used, the old value can remain.

Why it matters

Repeated uploads can become a mixture of old and new data.

Remediation being considered

For batch replacement, validate first, then replace only that tenant's slice atomically. For high-frequency loads, use staging plus a full UPSERT that updates existing rows.

Verification

Run the same upload twice; then correct one field and re-upload. Verify another tenant is untouched.

Research basis [R2] PostgreSQL INSERT documentation.

Issue 3: E-07 — guardian_consent_id vs guardian_consent_hash mismatch

Layer: Schema / mapper

Current problem

Different generators use different names for the same concept while the mapper expects one form.

QuickLoan example

> QuickLoan contains guardian_consent_id, but the mapper checks guardian_consent_hash, so a valid consent may look missing.

Why it matters

Naming mismatch can create false violations and broken joins.

Remediation being considered

Define one canonical field in a shared schema. Update all generators. During migration, support a legacy alias with a warning; required fields should fail loudly.

Verification

Validate every generator against the same schema.

Research basis [R4] Data-contract / Data Mesh literature.

Issue 4: E-08 — Mapper forces dpa_signed=False for internal systems

Layer: Mapper / rule engine

Current problem

The mapper overwrites uploaded DPA information for internal systems.

QuickLoan example

> QuickLoan uploads an internal system with dpa_signed=True; the mapper changes it to False and DPDP-016 can fire.

Why it matters

The mapper is making a compliance decision and destroying source information.

Remediation being considered

Preserve the uploaded value. Put applicability logic inside DPDP-016 and explicitly restrict the rule to the processor types it is meant to check.

Verification

Test internal, third-party, and sub-processor cases with true/false/unknown DPA values.

Research basis [R5] Rule/data-transformation separation; authoritative DPDP materials for exact legal scope.

Issue 5: E-09 — Two incompatible synthetic-data generator paths

Layer: Synthetic data

Current problem

The reviewed generators use conflicting assumptions, including one path that is hardcoded not to generate cross-border cases.

QuickLoan example

> One QuickLoan generator can create cross-border transfers; another always creates zero.

Why it matters

Test coverage and reproducibility depend on which script was used.

Remediation being considered

Consolidate into one profile-driven generator with explicit scenario rates and deterministic seeds.

Verification

Same profile + seed should reproduce the same dataset; changing one rate should change the expected scenario distribution.

Research basis [R6] Jordon et al., Synthetic Data — what, why and how?; [R7] Synthetic Data Vault.

Issue 6: E-01 — Denominator dilution

Layer: Risk scoring

Current problem

The current ratio uses affected_rows / total_rows, even when a rule only evaluates a small subset.

QuickLoan example

> QuickLoan has 50,000 transactions but 200 applicable consent records. If all 200 fail, 200/50,000=0.4% hides a 100% failure rate among applicable records.

Why it matters

Serious rule failures are diluted by unrelated rows.

Remediation being considered

Use violation_count / opportunity_count, where opportunity_count is the number of records actually evaluated by that rule. Store both counts.

Verification

Create a test where every applicable record violates the rule; expected rule rate is 100%.

Research basis [R8] FAIR materials support carefully defining the relevant event/frequency; the opportunity denominator is a project-specific design.

Issue 7: E-01 / severity — No severity floor for critical findings

Layer: Risk scoring

Current problem

A weighted aggregate can be pulled down by many compliant records even when a critical rule fires.

QuickLoan example

> QuickLoan violates a critical children's-data rule. The result should not become LOW merely because the dataset is large.

Why it matters

Numeric tier can contradict the seriousness of the strongest finding.

Remediation being considered

Apply an explicit project policy after aggregation: proposed floors are CRITICAL→minimum 65/HIGH; 3+ HIGH→minimum 45/MEDIUM; 10+ distinct rules→breadth adjustment. These are not CVSS-mandated numbers.

Verification

Create controlled one-critical, three-high, and ten-rule cases and validate the policy.

Research basis [R9] FIRST CVSS v3.1; used as conceptual severity/context guidance only.

Issue 8: E-06 — Fake XAI metrics: phi / total_shap

Layer: Explainability

Current problem

Rule signal weights are presented as SHAP values even though they are designer-defined constants.

QuickLoan example

> QuickLoan fires a rule with weight 0.8. Calling that SHAP=0.8 falsely suggests XGBoost calculated it.

Why it matters

The explanation is misleading.

Remediation being considered

Keep signal_contribution/total_signal_weight for deterministic rules. Compute real SHAP values for the XGBoost model and label the source explicitly.

Verification

Change a rule weight and verify only the rule contribution changes; separately verify model SHAP output.

Research basis [R10] Lundberg & Lee, A Unified Approach to Interpreting Model Predictions (NeurIPS 2017).

Issue 9: E-11 — fields_triggered is hardcoded / evidence is duplicated

Layer: Rule evidence

Current problem

fields_triggered is returned as an empty list while why_detected is used as narrative.

QuickLoan example

> QuickLoan violates a rule because consent_status='expired' and is_cross_border=True. The report should identify both fields.

Why it matters

Reviewers cannot trace the finding back to the exact triggering fields.

Remediation being considered

During signal evaluation, collect field names whenever signals fire and propagate them to fields_triggered. Keep narrative and evidence separate.

Verification

Test a two-signal violation and compare fields_triggered with the evaluation trace.

Research basis [R11] Audit-evidence principles such as ISA 500.

Issue 10: E-02 — SMOTE before cross-validation

Layer: ML training

Current problem

SMOTE is applied before the fold split, allowing information from validation data to influence synthetic training examples. The reviewed process also evaluates on the expanded training data.

QuickLoan example

> A rare QuickLoan CRITICAL example can influence synthetic samples before validation folds are separated.

Why it matters

Reported metrics can be overly optimistic.

Remediation being considered

Create a stratified holdout first. Put SMOTE inside an imbalanced-learn Pipeline with XGBoost and cross-validate the pipeline. Evaluate final performance only on the untouched holdout.

Verification

Confirm the holdout is never used by SMOTE/fit and that final metrics come only from it.

Research basis [R12] Kaufman et al., Leakage in Data Mining; [R13] scikit-learn leakage guidance; [R14] imbalanced-learn.

Issue 11: E-02 / metrics — Weak metrics for imbalanced multi-class risk

Layer: ML evaluation

Current problem

Accuracy and weighted F1 can hide poor performance on rare classes.

QuickLoan example

> A model predicting LOW for almost everything can look good overall while missing CRITICAL.

Why it matters

Model quality can look strong while high-risk detection is poor.

Remediation being considered

Report macro F1, MCC, per-class precision/recall/F1, and appropriate PR performance. Keep accuracy/weighted F1 secondary.

Verification

Include a confusion matrix and class-level results.

Research basis [R13] scikit-learn; metric selection is project policy.

Issue 12: E-03 — ML layer omitted from API response

Layer: API / reporting

Current problem

The orchestrator creates ml_layer but the API schema/response builder drops it.

QuickLoan example

> Audit data can say ML=CRITICAL while API says LOW.

Why it matters

Two decision records can disagree.

Remediation being considered

Add optional ml_layer, pass it through the response builder, and persist deterministic and ML outputs separately with model metadata.

Verification

Compare orchestrator, database, and API outputs for QuickLoan.

Research basis [R15] EDPB automated-decision guidance; [R16] NIST AI RMF.

Issue 13: E-05 — Unauthenticated rule-management endpoints

Layer: API security

Current problem

Rule approval can modify live rule definitions without the required authentication/role checks.

QuickLoan example

> An unauthorized caller could potentially change the rules used to judge QuickLoan and all tenants.

Why it matters

Changing rules changes the compliance logic itself.

Remediation being considered

Require authentication, explicit admin/reviewer roles, deny by default, and audit every approval/rejection.

Verification

Test anonymous, ordinary-user, reviewer, and admin access.

Research basis [R17] OWASP API Security Top 10 2023; [R18] OAuth 2.0 RFC 6749.

Issue 14: E-12 — Dynamic SQL identifier interpolation

Layer: Database security

Current problem

A table name is inserted into an SQL string using interpolation. Identifiers require a different safety mechanism from ordinary values.

QuickLoan example

> If primary_table is attacker-influenced, crafted input could alter query structure.

Why it matters

A compromised rule could become an SQL-injection path.

Remediation being considered

Whitelist allowed table names. For genuine dynamic identifiers use psycopg.sql.Identifier; keep normal parameter placeholders for values.

Verification

Reject malformed/non-whitelisted identifiers and test SQL metacharacters.

Research basis [R19] Psycopg SQL composition; [R20] CWE-89.

Issue 15: E-13 — Missing run-level versioning

Layer: Governance

Current problem

Persisted results do not identify the exact model, rules, schema, and code used.

QuickLoan example

> Six months later QuickLoan has a new model and rules; an old result cannot be reproduced.

Why it matters

Historical findings become difficult to audit or defend.

Remediation being considered

Persist model artifact/hash, rules version, schema version, and source-code commit in run_metadata.

Verification

Run QuickLoan under different artifact/rule/schema versions and confirm metadata changes.

Research basis [R16] NIST AI RMF; [R21] Model Cards; [R22] ML Technical Debt.

4. Cross-Cutting Design Rules

- Uploaded source data is authoritative unless a transformation is explicitly documented.
- Every tenant replacement operation must be scoped to that tenant and executed atomically.
- Generators, mapper, database, rules, and API should share one versioned canonical schema.
- Every fired signal should be traceable to a rule, field/value, and rule-definition version.
- Same generator version + profile + seed should reproduce the same dataset.
- Rule weights and ML SHAP values must remain separate and explicitly labeled.
- Persist model, rules, schema, generator/profile, and code versions needed to reproduce a run.

5. Implementation Sequence and Exit Conditions

1	Ingestion integrity	Reject/report missing references; fix tenant replacement; canonical field; preserve DPA values.	Database state is entirely source-backed and tenant-safe.
2	Generator reconciliation	One profile-driven generator; deterministic seeds; full scenario coverage.	Same profile + seed reproduces the dataset.
3	Clean regeneration + re-ingestion	Regenerate all test tenants and load into a clean database.	No manufactured/stale contamination remains.
4	Rule engine + evidence	Move applicability into rules; populate fields_triggered.	Every finding is traceable to signals and evidence.
5	Risk scoring	Opportunity denominators + approved severity policy; rerun tenants.	Scores are explainable rule-by-rule.
6	ML retraining	Holdout + fold-safe SMOTE + clean data + imbalance metrics.	Final model metrics come from untouched data.
7	API + explainability	Expose ml_layer; compute true SHAP; preserve evidence.	API, orchestrator, and persisted results agree.
8	Security + governance	RBAC/authentication, safe identifiers, run metadata.	Rule changes are protected and historical runs reproducible.

Critical dependency: Do not retrain the ML model before Phases 1–3 are complete and verified. Otherwise the new model can learn the same contaminated patterns again.

6. Verification Checklist

1. Missing consent references create no synthetic consent rows.
2. Missing third-party references create no synthetic system/DPA rows.
3. Identical tenant ingestion produces identical tenant state.
4. Corrected uploads replace old values without touching other tenants.
5. All generator paths produce canonical field names.
6. Same generator profile + seed reproduces the same dataset.
7. Internal DPA values are preserved by the mapper.
8. DPDP-016 evaluates only its declared processor types.
9. fields_triggered exactly matches the fields of fired signals.
10. Rule denominators represent the actual opportunity set.
11. Severity floors are documented as project policy and tested.
12. SMOTE is fitted only within training folds.
13. Final ML metrics come from an untouched holdout.
14. Macro F1/MCC/per-class results make rare classes visible.
15. Rule contribution is never labeled as SHAP.
16. API exposes deterministic and ML outputs separately.
17. Anonymous/unauthorized rule-management requests are denied.
18. Dynamic table names are whitelisted and safely composed.
19. Persisted results contain model/rules/schema/commit metadata.

7. Precision Notes / Caveats

Research principle vs project policy: research supports principles such as leakage prevention and truthful explainability. It does not establish project-specific thresholds such as 65/45.

CVSS is an analogy, not a DPDP scoring standard: it supports severity/context thinking but does not prescribe QuickLoan's thresholds.

FAIR is not the engine's formula: FAIR describes risk through frequency and magnitude of loss; the opportunity-based denominator is a project-specific design.

Legal context: GDPR/EDPB sources are governance references, not Indian law. DPDP legal claims require authoritative Indian sources.

Tenant replacement safety: validate first, then replace atomically. Never delete good data and only afterward discover that the new upload is invalid.

SHAP: exact implementation depends on the installed SHAP version and actual XGBoost model type, so it must be tested against the project's artifact.

8. Research & Reference List

[R1] Martin Kleppmann, Designing Data-Intensive Applications. O'Reilly, 2017.

[R2] PostgreSQL — INSERT / ON CONFLICT.
https://www.postgresql.org/docs/current/sql-insert.html

[R4] ThoughtWorks / Data Contracts and Data Mesh literature.

[R5] Authoritative DPDP legal materials plus project rule definitions.

[R6] Jordon et al., Synthetic Data — what, why and how? (2022).
https://arxiv.org/abs/2205.03257

[R7] Patki, Wedge & Veeramachaneni, The Synthetic Data Vault (DSAA 2016).
https://dblp.org/rec/conf/dsaa/PatkiWV16

[R8] FAIR Institute — Factor Analysis of Information Risk.
https://www.fairinstitute.org/

[R9] FIRST — CVSS v3.1 Specification.
https://www.first.org/cvss/v3-1/specification-document

[R10] Lundberg & Lee, A Unified Approach to Interpreting Model Predictions (NeurIPS 2017).
https://papers.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

[R11] ISA 500 / audit-evidence principles.

[R12] Kaufman et al., Leakage in Data Mining: Formulation, Detection, and Avoidance (ACM TKDD, 2012).
https://dblp.org/rec/journals/tkdd/KaufmanRPS12.html

[R13] scikit-learn — Common pitfalls / Data Leakage.
https://scikit-learn.org/stable/common_pitfalls.html

[R14] imbalanced-learn — Pipeline / SMOTE documentation.

[R15] European Data Protection Board — Automated individual decision-making and Profiling.
https://www.edpb.europa.eu/documents/guideline/automated-decision-making-and-profiling_en

[R16] NIST AI Risk Management Framework 1.0 (NIST AI 100-1, 2023).
https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10

[R17] OWASP API Security Top 10 2023 — API5 Broken Function Level Authorization.
https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/

[R18] IETF RFC 6749 — OAuth 2.0 Authorization Framework.
https://www.rfc-editor.org/info/rfc6749/

[R19] Psycopg — SQL composition and Identifier.
https://www.psycopg.org/psycopg3/docs/api/sql.html

[R20] CWE-89 — SQL Injection.

[R21] Mitchell et al., Model Cards for Model Reporting (FAccT 2019).
https://facctconference.org/2019/acceptedpapers.html

[R22] Sculley et al., Machine Learning: The High-Interest Credit Card of Technical Debt.
https://research.google.com/pubs/archive/43146.pdf

9. Final Recommendation

Implement the remediation as small, testable changes. After each phase, lock the verified behavior with regression tests before moving downstream. The goal is not simply to change QuickLoan's score; it is to make every result traceable to source data, rule logic, model behavior, and versioned artifacts.

> **Definition of success:** For a QuickLoan run, the system can clearly answer: what data was received; which rules fired and what fields/evidence triggered them; how the deterministic score was calculated; and, when ML is used, what the model predicted and which exact artifacts produced the result.